16.4 故障转移错误处理

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~400,000 tokens，输出 ~32,000 tokens（本章合计）

故障转移的核心问题是：什么样的错误应该触发回退？什么样的错误应该直接报告给用户？ OpenClaw 通过 `FailoverError` 类型和一套错误分类系统来精确控制故障转移的触发条件。

16.4.1 错误分类（`src/agents/failover-error.ts`）

FailoverError 类

`FailoverError` 是一个自定义错误类，携带了故障转移所需的全部上下文信息：

```typescript
// src/agents/failover-error.ts
export class FailoverError extends Error {
  readonly reason: FailoverReason;    // 失败原因分类
  readonly provider?: string;         // 提供者标识
  readonly model?: string;            // 模型标识
  readonly profileId?: string;        // Auth Profile ID
  readonly status?: number;           // HTTP 状态码
  readonly code?: string;             // 错误代码

  constructor(message: string, params: {
    reason: FailoverReason;
    provider?: string;
    model?: string;
    profileId?: string;
    status?: number;
    code?: string;
    cause?: unknown;
  }) {
    super(message, { cause: params.cause });
    this.name = "FailoverError";
    // ...赋值
  }
}
```

`FailoverReason` 枚举了所有可能的故障原因：

reason	含义	典型 HTTP 状态码
`billing`	计费错误（额度不足）	402
`rate_limit`	速率限制	429
`auth`	认证失败	401 / 403
`timeout`	请求超时	408
`format`	请求格式错误	400
`unknown`	未知错误	—

错误到故障原因的映射

`resolveFailoverReasonFromError` 函数负责将任意错误对象映射到 `FailoverReason`：

```ts
// src/agents/failover-error.ts
export function resolveFailoverReasonFromError(err: unknown): FailoverReason | null {
  // 1. 如果已经是 FailoverError，直接使用其 reason
  if (isFailoverError(err)) return err.reason;

  // 2. 根据 HTTP 状态码分类
  const status = getStatusCode(err);
  if (status === 402) return "billing";
  if (status === 429) return "rate_limit";
  if (status === 401 || status === 403) return "auth";
  if (status === 408) return "timeout";

  // 3. 根据错误代码分类（Node.js 网络错误）
  const code = getErrorCode(err);
  if (["ETIMEDOUT", "ESOCKETTIMEDOUT", "ECONNRESET", "ECONNABORTED"].includes(code)) {
    return "timeout";
  }

  // 4. 根据错误消息文本分类
  if (isTimeoutError(err)) return "timeout";
  return classifyFailoverReason(getErrorMessage(err));
}
```

分类逻辑的优先级是：已有标记 > HTTP 状态码 > 网络错误代码 > 错误消息文本。这确保了即使来自不同 SDK 的错误格式不统一，也能被正确分类。

超时检测

超时错误的检测尤其复杂，因为不同环境和 SDK 的超时错误表现形式各异：

```ts
// src/agents/failover-error.ts
const TIMEOUT_HINT_RE = /timeout|timed out|deadline exceeded|context deadline exceeded/
const ABORT_TIMEOUT_RE = /request was aborted|request aborted/i;

export function isTimeoutError(err: unknown): boolean {
  // 方式 1：错误名为 "TimeoutError"
  if (getErrorName(err) === "TimeoutError") return true;

  // 方式 2：错误消息包含超时关键词
  if (TIMEOUT_HINT_RE.test(getErrorMessage(err))) return true;

  // 方式 3：AbortError + 超时相关消息（区分用户主动取消和超时导致的取消）
  if (getErrorName(err) === "AbortError") {
    if (ABORT_TIMEOUT_RE.test(getErrorMessage(err))) return true;
    // 还检查 cause 和 reason 字段
    if (hasTimeoutHint(err.cause) || hasTimeoutHint(err.reason)) return true;
  }

  return false;
}
```

超时与 AbortError 的区分至关重要——用户主动 `/stop` 产生的 AbortError 不应触发故障转移，而超时导致的 AbortError（如某些 HTTP 库将超时表现为 abort）应该触发。

错误强制转换

普通错误可以被“提升”为 `FailoverError`：

```ts
// src/agents/failover-error.ts
export function coerceToFailoverError(
  err: unknown,
  context?: { provider?: string; model?: string },
): FailoverError | null {
  if (isFailoverError(err)) return err;

  const reason = resolveFailoverReasonFromError(err);
  if (!reason) return null;  // 无法分类的错误不能转换

  return new FailoverError(getErrorMessage(err), {
    reason,
    provider: context?.provider,
    model: context?.model,
    status: getStatusCode(err) ?? resolveFailoverStatus(reason),
    code: getErrorCode(err),
    cause: err,
  });
}
```

如果一个普通错误可以被分类为某种故障原因，它就会被包装为 `FailoverError` ，进入故障转移流程。如果无法分类（返回 `null` ），说明这是一个不可恢复的错误（如配置错误），应该直接上抛。

16.4.2 认证错误、计费错误、上下文溢出错误的检测与处理

除了通用的错误分类外，OpenClaw 还针对几种特定错误类型实现了专门的检测和恢复策略。

认证错误

认证错误通常发生在 API Key 失效、Token 过期或权限不足时。在 `runEmbeddedPiAgent` 中，认证错误触发 Auth Profile 轮换：

```ts
// 检测认证错误
const authFailure = isAuthAssistantError(lastAssistant);
if (authFailure) {
  await markAuthProfileFailure({ store: authStore, profileId: lastProfileId, reason: "a
  if (await advanceAuthProfile()) continue;  // 切换到下一个 Profile
}
```

计费错误

计费错误（HTTP 402）意味着用户的额度已用完。OpenClaw 对计费错误有特殊处理：

1. 标记 Profile 进入长冷却期（计费错误的退避时间更长）
2. 尝试切换到其他 Profile（可能有不同的计费账号）
3. 如果所有 Profile 都是计费错误，尝试回退到其他模型
4. 最终向用户展示专用的计费错误消息

上下文溢出错误

上下文溢出（Context Overflow）发生在对话历史超过模型的上下文窗口时。这种错误的恢复策略是自动压缩会话历史：

```
// src/agents/pi-embedded-runner/run.ts（简化版）
if (isContextOverflowError(errorText)) {
  if (overflowCompactionAttempts < MAX_OVERFLOW_COMPACTION_ATTEMPTS) {  // 最多 3 次
    overflowCompactionAttempts++;
    const compactResult = await compactEmbeddedPiSessionDirect({ ... });
    if (compactResult.compacted) {
      continue;  // 压缩成功，重试
    }
  }
  // 压缩失败或已达最大尝试次数
  return { payloads: [{ text: "Context overflow: prompt too large...", isError: true }]
}
```

自动压缩通过总结对话历史的早期部分来释放 Token 空间。如果连续 3 次压缩都无法解决溢出问题（可能是单条消息本身就超过了上下文窗口），则向用户报告错误。

消息角色顺序错误

某些 LLM API 要求消息角色严格交替（user → assistant → user → ...）。如果历史中出现了连续的同角色消息，会导致角色顺序错误：

```
if (/incorrect role information|roles must alternate/i.test(errorText)) {
  return {
    payloads: [{
      text: "Message ordering conflict - please try again. " +
            "If this persists, use /new to start a fresh session.",
      isError: true,
    }],
  };
}
```

角色顺序错误不触发故障转移（因为换个模型也一样会失败），而是直接提示用户重试或开启新会话。

图片大小错误

当用户发送的图片超过模型的限制时：

```
const imageSizeError = parseImageSizeError(errorText);
if (imageSizeError) {
  return {
    payloads: [{
      text: `Image too large for the model (max ${imageSizeError.maxMb}MB). ` +
            "Please compress or resize the image and try again.",
      isError: true,
    }],
  };
}
```

图片大小错误也不触发故障转移——用户需要压缩图片后重新发送。

16.4.3 故障转移日志与用户可见的错误消息

故障转移日志

每次故障转移尝试都会被记录，包括：

- 尝试的提供者和模型
- 错误消息和分类原因
- HTTP 状态码和错误代码

```
// src/agents/model-fallback.ts (onError 回调)
await params.onError?.({
  provider: candidate.provider,
  model: candidate.model,
  error: normalized,
  attempt: i + 1,
  total: candidates.length,
});
```

错误描述

`describeFailoverError` 函数将任意错误对象转换为结构化的错误描述：

```
// src/agents/failover-error.ts
export function describeFailoverError(err: unknown): {
  message: string;
  reason?: FailoverReason;
  status?: number;
  code?: string;
} {
  if (isFailoverError(err)) {
    return { message: err.message, reason: err.reason, status: err.status, code: err.co
  }
  return {
    message: getErrorMessage(err) || String(err),
    reason: resolveFailoverReasonFromError(err) ?? undefined,
    status: getStatusCode(err),
    code: getErrorCode(err),
  };
}
```

全部失败时的错误汇总

当所有候选模型都失败时，系统汇总所有尝试的错误信息：

```
// src/agents/model-fallback.ts
const summary = attempts
  .map((attempt) =>
    `${attempt.provider}/${attempt.model}: ${attempt.error}` +
    `${attempt.reason ? ` (${attempt.reason})` : ""}`
  )
  .join(" | ");
throw new Error(`All models failed (${attempts.length}): ${summary}`);
```

例如，一个典型的全部失败错误消息可能是：

```
All models failed (3):
  anthropic/claude-sonnet-4-20250514: Rate limit exceeded (rate_limit) |
  openai/gpt-4o: Authentication failed (auth) |
  google/gemini-2.5-pro: Request timeout (timeout)
```

这个汇总消息清楚展示了每个候选模型的失败原因，帮助用户或管理员快速定位问题。

用户可见的错误消息

面向用户的错误消息经过精心设计，避免暴露内部细节：

错误类型	用户看到的消息
上下文溢出	"Context overflow: prompt too large for the model. Try again with less input or a larger-context model."
角色顺序冲突	"Message ordering conflict - please try again. If this persists, use /new to start a fresh session."
图片过大	"Image too large for the model (max NMB). Please compress or resize the image and try again."
计费错误	专用的 `BILLING_ERROR_USER_MESSAGE` 常量
通用超时	"LLM request timed out."
通用认证失败	"LLM request unauthorized."
通用速率限制	"LLM request rate limited."

所有用户可见的错误消息都附带了**可行的操作建议**——告诉用户接下来可以做什么，而不是仅仅报告问题。

本节小结

1. **`FailoverError`** 是故障转移系统的核心类型，携带了故障原因（`reason`）、提供者、模型、HTTP 状态码等完整上下文。
2. **错误分类**按四级优先级进行：已有标记 → HTTP 状态码 → 网络错误代码 → 错误消息文本，确保不同 SDK 的错误都能被正确分类。
3. **超时与 AbortError 的区分**是关键——用户主动取消不触发回退，超时导致的中止触发回退。
4. **特定错误类型**（认证、计费、上下文溢出、角色顺序、图片大小）各有专门的检测和恢复策略，不能一概而论。
5. 全部失败时的**错误汇总**展示了完整的失败链，用户可见的错误消息附带可行的操作建议。